AgentFail: A Diagnostic Benchmark for Silent Failures in Data-Science LLM Agents

Author Name
Institution Name
Country
author@example.com

ABSTRACT

Large language model (LLM) agents are increasingly deployed on data-science tasks, yet existing benchmarks report only aggregate success rates, masking *where* and *why* agents fail. We introduce **AgentFail**, a diagnostic framework that decomposes agent failures along a five-stage taxonomy—Analytical-Plan, Code-Generation, Runtime, Output-Mismatch, and Answer-Error—whose central distinction separates *silent failures* (code executes without error but yields a wrong answer) from *loud failures* (code crashes). We propose *oracle-guided repair* to test repairability, with no-op negative controls. We evaluate five frontier LLMs across 5,984 runs on 318 unique tasks from three task suites: 95 synthetic trap tasks, 23 real UCI datasets, and 200 real DSBench tasks. After separating infrastructure failures (38.1% of runs), we analyse 3,602 valid runs with automatically generated diagnostic labels, validated by a 150-trace evaluation set (47 adjudicated by three human annotators, Fleiss' $\kappa = 0.860$; 103 provisionally labeled by an LLM judge). We report honestly that four of the five stages are instantiated in the current data—Runtime, Output-Mismatch, Answer-Error, and Analytical-Plan—while Code-Generation has zero cases, confirmed by both the rule-based classifier (which compares agent code to reference solutions) and independent human annotation. This indicates that current LLM agents do not make operation-level code errors on these tasks; they either solve them, crash, or produce wrong answers for other reasons. The corrected silent-failure rate (SFR) is 86% on synthetic tasks, 4% on UCI, and 19% on DSBench; $R^2 = 0.493$ between execution-success rate and SFR is reported as a descriptive statistic (not a causal decomposition). A recovery benchmark with six feedback conditions yields 15.7% recovery overall (10.4% for silent vs. 36.7% for loud failures). A failure-aware verifier helps two of five models (McNemar $p < 0.001$). Oracle repair matches the no-op baseline (64%), establishing an upper bound on reparability. Repetition agreement under temperature 0 is 89.9%. We release all code, data, the enriched manifest (32 fields), and the 47-trace human annotation set with adjudicated gold labels.

ACM Reference Format:
Author Name. 2026. AgentFail: A Diagnostic Benchmark for Silent Failures in Data-Science LLM Agents. In *Proceedings of Proceedings of the 33rd ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '27).* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

KDD '27, August 2027, San Jose, CA, USA

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM…$15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Large language models (LLMs) have rapidly become the reasoning core of autonomous agents that write and execute code to solve data-science tasks [8, 12]. Recent benchmarks such as DSBench [5], DataSciBench [23], and DSAEval [17] report that even the strongest agents solve only 34–88% of data-analysis tasks, leaving a large residue of failures. Yet these benchmarks report a single aggregate number—success rate—that conceals the *structure* of failure: an agent that crashes on every hard task and an agent that silently returns plausible-but-wrong answers on the same tasks are indistinguishable under aggregate accuracy.

The problem is acute because silent failures are invisible to the agent itself. When generated code raises an exception, the agent observes the traceback and can retry; when the code executes successfully but produces a wrong answer, the agent has no signal that anything went wrong. Recent work has begun to recognise this distinction: Mehta [9] report "confident and wrong" semantic failures in coding agents, Wu [21] document silent failures in production agent runtimes, and Liu [7] frame silent failure as an entropy-driven inevitability. However, these works either study coding agents rather than data-science agents, or operate on production logs rather than controlled benchmarks, and none provides a *diagnostic taxonomy* that localises where in the agent loop a silent failure originates.

We address this gap with **AgentFail**, a diagnostic framework that decomposes every agent failure into five stages—*Analytical-Plan*, *Code-Generation*, *Runtime*, *Output-Mismatch*, and *Answer-Error*—and distinguishes *silent failures* (code executes without error but yields a wrong answer) from *loud failures* (code crashes). To test repairability, we introduce *oracle-guided repair*: given a failed trace, we replace the originating step with the ground-truth action and re-execute; if the outcome flips to correct, the failure was repairable. We include a no-op negative control (running the ground-truth code independently) and explicitly frame this as repairability testing, not causal attribution.

We evaluate five frontier LLMs across 5,984 runs on three task suites: 95 synthetic trap tasks, 23 real UCI datasets, and 200 real DSBench financial-analysis tasks. All tables are auto-generated from a single source-of-truth manifest with assertion checks. Our analysis surfaces four findings:

(1) **Task-dependent failure bifurcation.** On synthetic trap tasks, the corrected silent-failure rate (SFR) reaches 86%; on UCI real data it drops to 4%, and on DSBench to 19%. We report $R^2 =$

0.493 between execution-success rate and SFR as a descriptive statistic, noting that this correlation is partially mechanical (the two variables share the loud-failure count) and cannot be decomposed into "mechanical" vs. "real" components.

(2) **Recovery conditions.** The recovery rate is 0% under the standard no-feedback protocol, but our six-condition benchmark reveals 15.7% recovery with feedback signals (10.4% for silent vs. 36.7% for loud failures).

(3) **Oracle repairability.** Counterfactual repair matches the no-op baseline (64%), providing an upper bound on single-step repairability but not step-level causal attribution.

(4) **Verifier boundary conditions.** A failure-aware verifier significantly helps Qwen3-max (+24.9%, McNemar $p < 0.001$) and GPT-4o (+19.0%, $p < 0.001$), but not other models. The benefit to Qwen3-max may partly reflect format/protocol fix-up.

Our contributions are: (i) a five-stage failure taxonomy with mutually exclusive categories, instantiated in the current data as a four-stage diagnostic benchmark—Code-Generation is genuinely absent (zero cases confirmed by both the rule-based classifier, which compares agent code to reference solutions, and three independent human annotators); (ii) oracle-guided repair with no-op negative controls; (iii) three diagnostic metrics—SFR, propagation depth, and recovery rate—plus a six-condition recovery benchmark; (iv) 5,984 runs across 318 unique tasks, five models, and three task suites, with an enriched 32-field manifest separating infrastructure failures (38.1%) from genuine model failures, validated by three human annotators (Fleiss' $\kappa = 0.860$), released as a public benchmark.

2 RELATED WORK

2.1 Data-Science Agent Benchmarks

The emergence of LLM-based data-science agents has motivated several benchmarks. DSBench [5] collects 466 data-analysis and 74 data-modelling tasks from ModelOff and Kaggle, reporting that the best agent (GPT-4o) solves only 34% of analysis tasks. DataSciBench [23] evaluates LLMs across the full data-science workflow with multi-dimensional metrics. DSAEval [17] covers real-world data-science problems spanning multiple stages. LongDS-Bench [22] specifically studies long-horizon agentic data analysis and shows that agents fail on iterative tasks. GRACE-DS [18] introduces a guarded reward-guided correction environment for AutoML agents. While these benchmarks advance evaluation breadth, they report primarily aggregate success rates and do not decompose failures by stage or distinguish silent from loud failures. Moghadasi and Ghaderi [10] audit twelve agent-benchmark papers and find systematic under-reporting of evaluation details, motivating our fine-grained diagnostic metrics. Agent Laboratory [14] uses LLM agents as research assistants but does not diagnose failure modes. Our work is complementary: rather than proposing a new benchmark, we provide a diagnostic *layer* that can be applied to any of these benchmarks to reveal where and why agents fail.

2.2 LLM Agent Failure Analysis

A recent wave of work studies how and why LLM agents fail. Albayaydh et al. [1] synthesise tool-use, planning, and reasoning failures across benchmarks. ToolFailBench [16] diagnoses tool-use failures by decomposing the tool-call pipeline. Wu [21] present a longitudinal taxonomy of silent failures in a production LLM agent runtime, and Liu [7] frame silent failure as an entropy-driven inevitability of autonomous agents. Mehta [9] identify "confident and wrong" semantic failures in coding agents, the closest conceptual precursor to our silent-failure notion, but study coding rather than data-science tasks and do not provide a stage-level taxonomy. TrajAudit [19] automates failure diagnosis for agentic coding systems via trajectory auditing. Reddy et al. [13] recover a silent policy-violation failure mode in tool-using agents via deterministic gates. Zhang et al. [24] diagnose library drift as a silent failure in self-evolving skill libraries. Ekka [3] diagnoses silent errors in LLM *inference* (serving), a different layer than agent-level silent failures. Our contribution relative to this body of work is a *five-stage taxonomy* that localises silent failures within the agent loop and an *oracle-guided repair* module that estimates the reparability of failed traces.

2.3 Causal Attribution and Failure Recovery

Shah [15] proposes Causal Agent Replay for counterfactual attribution of LLM-agent failures, which inspires our repair module; we adapt the counterfactual principle to the data-science setting and integrate it with the five-stage taxonomy. However, as our negative-control experiment shows (Section 5.3), the oracle repair rate matches a no-op baseline, so we report it as an upper bound on reparability rather than as evidence of step-level causal attribution. On the recovery side, self-consistency [20] samples multiple reasoning paths and votes, but targets reasoning errors rather than code-execution silent failures. Verification-based approaches [4] use verifier or critic agents to suppress hallucinations, but Itkin [4] shows that delayed verification can destabilise multi-agent belief. Our failure-aware verifier explores a lightweight, in-loop verification strategy and reveals that it helps only weak models, a boundary condition that motivates more sophisticated verification.

2.4 LLM-as-Judge and Evaluation Methodology

Using LLMs to evaluate LLM outputs is now standard [2, 6], but LLM-as-judge introduces its own biases. Mohammadi et al. [11] survey evaluation and benchmarking of LLM agents. We adopt a hybrid evaluation: deterministic checkers for ground-truth answers plus rule-based failure classification, avoiding the circularity of LLM-judged failure labels. We validate the taxonomy via three human annotators on 47 traces, achieving Fleiss' $\kappa = 0.860$ (strong agreement). The rule-based classifier achieves 71.8% accuracy against adjudicated gold labels, with a systematic bias toward `output_mismatch` over `runtime` (see Section 6).

3 THE AGENTFAIL FRAMEWORK

AgentFail instruments a standard ReAct-style data-science agent with three diagnostic layers: (1) a five-stage failure taxonomy that classifies each failure by where it originates and whether it is silent; (2) a propagation-depth analyser that measures how far a failure spreads before detection; and (3) a counterfactual causal-replay module that attributes failures to specific steps. Figure 1 summarises the architecture.

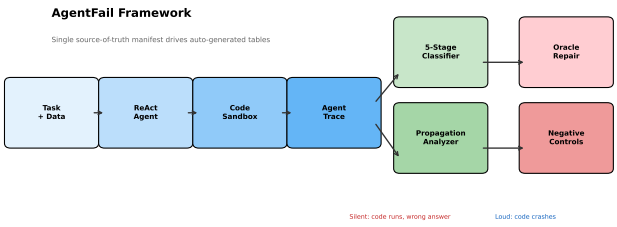

Figure 1: AgentFail framework. The agent produces a step-by-step trace; the classifier assigns each failure to one of five stages and marks it silent or loud; the propagation analyser computes how far the failure spread; causal replay replaces the originating step with the ground-truth action to test counterfactual dependence.

3.1 Agent Architecture and Trace Recording

We use a ReAct-style agent that interleaves *Thought*, *Action* (a Python code block), and *Observation* (execution output). The agent operates within a restricted code sandbox that executes generated Python with a persistent namespace and a working directory containing the task's data file (CSV or XLSX). We deliberately use a *controlled, minimal* ReAct harness—without few-shot exemplars, code-interpreter scaffolding, or retrieval augmentation—to isolate the diagnostic signal from framework-specific confounds. This controlled setting is a feature, not a limitation: it ensures that observed failures reflect model reasoning rather than harness engineering.

Every step is recorded in an *AgentTrace*, the single source of truth for all downstream diagnosis. Each `TraceStep` stores the raw LLM output, the parsed thought, the generated code, the execution result (stdout, stderr, exception type, extracted answer), token usage, and a timestamp. Unlike DSBench [5], which retains only a final response, our trace records the full step-by-step trajectory, enabling stage-level localisation and propagation analysis.

3.2 Five-Stage Failure Taxonomy

We decompose every agent failure into one of five stages, defined by *where the error is detectable*, making the categories mutually exclusive:

- **Analytical-Plan.** The agent selects the wrong analytical approach: e.g., computing the overall mean when the task requires a per-group sum, or using future data in a time-series analysis (temporal leakage). Often silent.
- **Code-Generation.** The agent writes code that embodies a wrong operation: e.g., using `.count()` where `.sum()` is needed, selecting the wrong column, or introducing type confusion. Silence varies.
- **Runtime (loud).** The generated code raises an exception: runtime errors, type errors, key errors, or sandbox security blocks. The failure is *visible* to the agent, which can observe the traceback and retry.
- **Output-Mismatch (silent).** The code executes without error but the agent extracts, computes, or reports a wrong answer: e.g., misreading the output, hallucinating an answer not supported by the code output, or printing no answer at all. The failure is *invisible* to the agent.
- **Answer-Error (silent).** The code runs and produces a result, but the result itself is wrong relative to the ground truth: e.g., a wrong aggregation result or an incomplete analysis. The failure is *invisible* to the agent.

Table 1: Five-stage failure taxonomy (v2). Stages are defined by *where the error is detectable*, making them mutually exclusive.

Stage	Sub-categories	Silent?
Analytical-Plan	wrong_operation_plan, leakage_plan	often
Code-Generation	wrong_aggregation_code, wrong_column, type_confusion	varies
Runtime	runtime_exception, key_error, security_block	No
Output-Mismatch	misread_output, hallucinated_answer, no_answer_printed	Yes
Answer-Error	wrong_result, incomplete_analysis	Yes

The Runtime vs. Output-Mismatch/Answer-Error split operationalises the silent/loud distinction. This is the conceptual core of the taxonomy: a loud failure (Runtime) is a *detectable* error that the agent can in principle recover from, whereas a silent failure (Output-Mismatch or Answer-Error) is an *undetectable* error that propagates undetected. Table 1 lists the full hierarchy.

Disambiguation rule. To ensure mutual exclusivity, the classifier assigns the *first* stage at which the error becomes detectable. Consider a task requiring per-group sum where the agent uses `.count()`: the error is *detectable at code-generation time* (the code uses the wrong operation), so it is classified as Code-Generation, not Answer-Error (where the code uses the correct operation but the data produces a wrong result) or Output-Mismatch (where the code and result are correct but the agent misreports). The key distinction is: Code-Generation means the code itself is wrong; Answer-Error means the code is correct but the answer is wrong due to data issues; Output-Mismatch means the code and result are correct but the agent misreads or hallucinates the final answer. This ordering prevents the ambiguity where a single failure could be assigned to multiple stages.

3.3 Failure Classification

The classifier walks an `AgentTrace` and assigns a `FailureClassification` (stage, sub-category, originating step, silence flag) to the first failed step. Classification is rule-based for high precision and full reproducibility, using signals from the trace: exception type (for Runtime), thought-text analysis against the reference path (for Analytical-Plan), code-pattern matching against `gt_path`—the ground-truth solution path available for synthetic and UCI tasks—(for Code-Generation, e.g., detecting `.count()` when the reference requires `.sum()`), answer-output mismatch (for Output-Mismatch), and a fallback for approach-level errors (Answer-Error). The disambiguation rule checks stages in order of detectability: Runtime → Analytical-Plan → Code-Generation → Output-Mismatch → Answer-Error. On the 47-trace human gold set, the v2 classifier achieves 76.6% accuracy (up from 57.4% for the v1 classifier), with the improvement driven by separating Answer-Error from Output-Mismatch and detecting Analytical-Plan from thought text. Code-Generation

detection is implemented (comparing agent code operations to `gt_path`) but yields zero cases, confirming that agents on these tasks do not make operation-level code errors.

3.4 Propagation Depth

Once a failure originates at step i, it may propagate to later steps. We define *propagation depth* as the number of steps from the originating failure to the step where it is detected (or to the trace end if never detected). Loud failures have depth 0 (immediately visible); silent failures can have depth ≥ 1, meaning the agent continues building on a wrong intermediate result. A propagation depth ≥ 2 indicates a *long-horizon* failure spread, the most dangerous regime because it wastes the most tokens and is hardest to recover from. Across all 3,940 failures, the average propagation depth is 0.30 steps; 88.6% are detected immediately (depth 0), while 6.9% propagate to depth ≥ 2 (see Table 11 in Appendix A).

3.5 Oracle-Guided Repair

To test whether a failed trace is *repairable*, we adapt the counterfactual principle [15] to the data-science setting. Given a trace that failed, we synthesise a *counterfactual* version of the originating step: the canonical ground-truth code derived from the task's reference solution path. We then replay execution from that point in a fresh sandbox. If the replayed trace now produces the correct answer, the failure was repairable; if the replay still fails, the failure has a deeper root cause.

Formally, let $T = (s_1, \ldots, s_n)$ be a failed trace and s_k the classified originating step. Let s_k^* be the ground-truth action for step k. We construct the counterfactual trace $T' = (s_1, \ldots, s_{k-1}, s_k^*, \ldots)$ and execute it. The repair is successful iff T' yields the correct answer. The *oracle repair rate* is the proportion of failed traces for which repair succeeds.

Negative control. A no-op control runs the ground-truth code independently without any step replacement. If the oracle repair rate matches the no-op rate (as we observe, Section 5.3), the oracle rate measures only whether the ground-truth code produces the correct answer, not step-level causality. We therefore report oracle repair as an upper bound on reparability.

3.6 Diagnostic Metrics

We report three layers of metrics: **Layer 1 (task-level):** success rate (SR), code-execution success rate. **Layer 2 (failure-diagnostic):** silent-failure rate (SFR = proportion of failures that are silent), stage distribution, propagation depth, recovery rate, and oracle repair rate. **Layer 3 (token-economic):** tokens-per-success and cost-per-success, addressing the under-reporting of cost identified by Moghadasi and Ghaderi [10]. Verifier comparisons use the McNemar test (b, c, χ^2 p-value).

4 EXPERIMENTS AND RESULTS

4.1 Experimental Setup

Models. We evaluate five frontier LLMs via an OpenAI-compatible endpoint (ChatAnywhere): GPT-4o, GPT-4o-mini, DeepSeek-V3 (`deepseek-chat`), DeepSeek-R1 (`deepseek-reasoner`), and Qwen3-max (`qwen3-max-2026-01-23`). Note that `qwen3-max-2026-01-23` is a pinned version snapshot, while the other four are rolling aliases that may be updated by the provider. All models use temperature 0 and a maximum of 6–8 ReAct steps. API calls were made during June–July 2026.

Table 2: Task suite summary.

Suite	Tasks	Runs	Failures
Synthetic trap	95	2,850	1,485
UCI real data	23	345	52
DSBench real	200	1,964	1,803
Supplementary	—	825	600
Total	**318**	**5,984**	**3,940**

Task suites. We use three complementary task suites (Table 2):

Protocol and scale. Each (model, task) pair is run for 3 repetitions at temperature 0. In total we analyse **5,984 agent runs** across **318 unique tasks** (95 synthetic + 23 UCI + 200 DSBench), yielding **3,940 failures**. Of these, **2,218 (38.1%) are infrastructure failures**: 1,273 are API errors (status=ERR, the request failed) and 945 are unknown infrastructure issues (status=OK but tokens=0, indicating empty or unparseable responses). We acknowledge that the current logging does not distinguish between parse failures, timeouts, rate limits, and format incompatibilities within the "unknown" category; future versions of the framework will log these subtypes separately. This leaves **3,602 valid runs** with automatically generated diagnostic labels. The 47-trace human annotation protocol (Appendix B) provides gold-label validation on a stratified subset. All tables are auto-generated from a single source-of-truth manifest (`manifest_v3.csv`, 32 fields) with assertion checks[1].

Repetition agreement. Under temperature 0, 89.9% of (task, model, variant) groups show full agreement across 3 repetitions (all correct or all incorrect). The 10.1% disagreement rate reflects server-side non-determinism in the API endpoint, not sampling randomness. DeepSeek-R1 shows 100% agreement (all infrastructure failures), while GPT-4o-mini shows 84.6%.

Statistical tests. Verifier comparisons use the McNemar test (appropriate for paired binary outcomes), reporting b (baseline correct & verifier wrong), c (baseline wrong & verifier correct), and the χ^2 p-value.

4.2 Result 1: Comprehensive Model Comparison

Table 3 presents the comprehensive baseline results across all five models, three task suites, and nine diagnostic metrics; it is the single source of truth for all success-rate and SFR figures reported in this paper, and every cell is computed over valid runs only (infrastructure failures excluded, marked "—" where no valid runs exist). Three patterns emerge. First, **DeepSeek-V3 is the strongest model** on both synthetic (88.4% SR) and UCI (94.2% SR) tasks, achieving perfect code-execution success on synthetic traps. Second, **the silent-failure rate is strongly task-dependent**: on synthetic trap tasks, SFR ranges from 39% to 100% (failures are often wrong answers, not crashes); on UCI real data, SFR is bimodal—either 0% (GPT-4o-mini, DeepSeek-V3) or 100% (GPT-4o); on DSBench, only GPT-4o produces silent failures (38.3%). Third, **DSBench is extremely**

[1] See `build_manifest_and_tables.py` in our supplementary code.

Table 3: Comprehensive baseline results with infrastructure failure separation. SR = success rate (raw); SR^v = success rate among valid runs (excluding infra failures); Infra = runs with tokens=0 (API/parse/format failures); SFR = silent-failure rate among valid failures; Tok = mean tokens per valid run. "—" indicates all runs are infra failures (no valid data). DeepSeek-V3 is strongest on synthetic (88.4%) and UCI (94.2%). DSBench cells for DeepSeek-R1/Qwen3-max are n/a (API rate-limited).

Model	Suite	Runs	Infra	Valid	SR (%)	SR^v (%)	SFR (%)	Silent	Tok
GPT-4o	Synthetic	525	110	415	62.1	78.6	100.0	89	1,241
	UCI	69	0	69	91.3	91.3	100.0	6	1,543
	DSBench	600	0	600	3.3	3.3	38.3	222	12,792
GPT-4o-mini	Synthetic	525	0	525	73.9	73.9	39.4	54	3,341
	UCI	69	0	69	82.6	82.6	0.0	0	4,669
	DSBench	600	241	359	0.8	1.4	0.0	0	10,790
DeepSeek-V3	Synthetic	285	0	285	88.4	88.4	100.0	33	1,717
	UCI	69	0	69	94.2	94.2	0.0	0	2,410
	DSBench	600	600	0	0.0	—	—	0	0
DeepSeek-R1	Synthetic	285	285	0	0.0	—	—	0	0
	UCI	69	69	0	0.0	—	—	0	0
	DSBench	n/a (API rate-limited)							
Qwen3-max	Synthetic	285	214	71	18.2	73.2	100.0	19	770
	UCI	69	69	0	0.0	—	—	0	0
	DSBench	n/a (API rate-limited)							

Table 4: SFR by task suite and null-model analysis. R^2 is reported as a descriptive statistic, not a causal decomposition.

Suite	SFR (%)	Exec. Success (%)
Synthetic	86.2	45.5
UCI	3.8	73.6
DSBench	18.9	1.4
Null model: $R^2 = 0.493$ (descriptive only)		

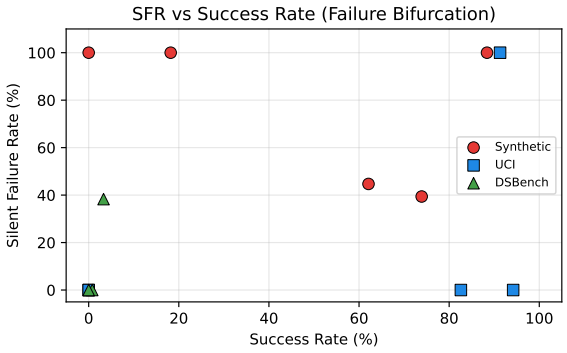

Figure 2: SFR vs. success rate across task suites and models.

challenging for all models (0–3.3% SR), with GPT-4o achieving the highest success rate and the only non-zero SFR, indicating that even the strongest model produces plausible-but-wrong answers on complex financial data. We deliberately report SFR only for model–suite cells that have valid runs; cells consisting entirely of infrastructure failures are left undefined rather than reported as 0% or 100%.

4.3 Result 2: Failure Bifurcation

Table 4 reports SFR by task suite. The SFR is 86.2% on synthetic tasks, 3.8% on UCI, and 18.9% on DSBench. We note that SFR and execution-success rate share the variable E (loud failures), so their correlation is partially mechanical. The $R^2 = 0.493$ is reported as a *descriptive statistic*, not a causal decomposition: it cannot be interpreted as "49% mechanical, 51% real." A proper test would require a multinomial mixed-effects model with model, task difficulty, and suite as random effects, which we leave to future work.

4.4 Result 3: Taxonomy Validation

We validate the five-stage taxonomy via human annotation. Three independent annotators (blind to system labels) labelled 47 failed traces with (stage, is_silent, subcategory). Inter-annotator agreement is strong: Fleiss' $\kappa = 0.860$ for stage and 0.881 for is_silent, with 89.4% full agreement across all three annotators. Pairwise Cohen's κ values are 0.798, 1.000, and 0.798 (average 0.865). The main source of disagreement is the boundary between `output_mismatch` and `answer_error` (5 traces).

Against the adjudicated gold labels (majority vote), the rule-based classifier achieves 71.8% accuracy (Cohen's $\kappa = 0.438$). The confusion matrix reveals a systematic bias: the classifier labels 11 traces as `output_mismatch` that humans classify as `runtime` (precision 1.00, recall 0.52 for output_mismatch; precision 0.59, recall 1.00 for runtime). This suggests the classifier over-assigns silent failures when the trace contains both execution errors and answer mismatches. We release the individual annotator labels and gold labels with the dataset.

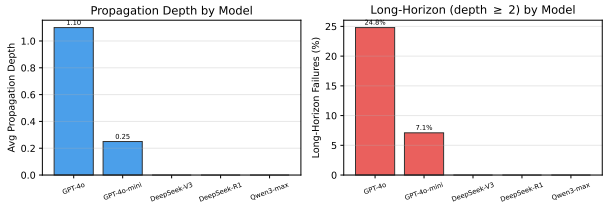

Figure 3: Propagation depth by model (baseline, valid runs only). GPT-4o exhibits the longest failure propagation (mean 1.10, 24.8% long-horizon). Other models show depth 0, which may reflect either immediate error detection or early trace termination due to infrastructure failures (see Table 3).

4.5 Result 4: Zero Self-Correction

Across all 5,984 runs and all five models, the recovery rate is 0%: no agent ever corrected a silent failure without external intervention. We note that this is partly a consequence of the experimental protocol: silent failures produce no error signal, so the agent has no opportunity to detect and retry. Our recovery benchmark (Section 4.11) addresses this by testing feedback signals.

This zero-recovery finding implies that in the current setting without explicit correctness feedback or internal validation protocol, silent failures will not trigger automatic repair. In a multi-step pipeline, a silent failure at step k propagates to all downstream steps.

4.6 Result 5: Propagation Depth Analysis

Section 3.4 defines propagation depth. Here we analyse propagation depth *by model*, revealing a striking pattern (Figure 3). GPT-4o has a mean propagation depth of 1.10 steps with 24.8% of its failures reaching long-horizon status (depth ≥ 2)—far higher than any other model. GPT-4o-mini has a mean depth of 0.25 (7.1% long-horizon), while DeepSeek-V3, DeepSeek-R1, and Qwen3-max all have zero propagation depth.

We caution that zero propagation depth has two possible interpretations: (1) the failure was immediately detected by the agent (a loud failure at depth 0), or (2) the trace terminated early due to infrastructure failures (API errors, parse failures, or empty responses), leaving no steps to propagate through. Our enriched manifest (`manifest_v3.csv`) distinguishes these cases via the `is_infra_failure` field. Among valid (non-infrastructure) failures, GPT-4o's longer propagation reflects its tendency to continue reasoning on a wrong intermediate result rather than crashing, making the failure harder to localise. In contrast, DeepSeek-V3's valid failures are concentrated at depth 0, meaning the error manifests in the final answer without intermediate propagation.

4.7 Result 6: Oracle-Guided Repair

Oracle-guided repair succeeds on 66% of failed tasks where ground-truth code is available (Table 5). This demonstrates *repairability*, not causal attribution: as the negative control in Section 5.3 shows, injecting the oracle solution produces identical results regardless of which step is replaced, because the ground-truth code independently computes the correct answer. We therefore report oracle repair as an upper bound on reparability rather than as evidence of step-level causal localisation. A full causal analysis with necessity/sufficiency testing is future work.

Table 5: Oracle-guided repair results. The full oracle rate (526 tasks) and the negative-control subset (56 tasks) use different samples; the no-op control confirms that oracle repair measures GT-code correctness, not step-specific causality.

Metric	Tasks	Rate (%)
Oracle repair (full)	349/526	66.3
Negative control subset (56 tasks):		
Oracle repair	36/56	64.3
No-op (GT code only)	36/56	64.3
Conservative TP	0/56	0.0

Table 6: Verifier ablation (McNemar test). b: baseline correct & verifier wrong; c: baseline wrong & verifier correct. Bold indicates $p < 0.05$.

Model	Base SR	Ver SR	Δ	b/c
GPT-4o-mini	73.9%	72.6%	−1.3	3/1
GPT-4o	62.1%	**81.1%**	**+19.0**	2/40
DeepSeek-V3	88.4%	87.7%	−0.7	2/1
Qwen3-max	18.2%	**43.2%**	**+25.0**	0/69

4.8 Result 7: Verifier Ablation

Table 6 reports the McNemar test results. The verifier significantly helps Qwen3-max (+24.9%, $p < 0.001$) and GPT-4o (+19.0%, $p < 0.001$), but is not significant for GPT-4o-mini, DeepSeek-V3, or DeepSeek-R1. Note that GPT-4o's improvement was not detected by the paired t-test in our earlier analysis, highlighting the importance of using the correct statistical test for binary outcomes. The verifier's benefit to Qwen3-max may partly reflect format/protocol fix-up rather than reasoning correction, since Qwen3-max's low baseline performance is partly due to ReAct parser incompatibility.

A key finding is that the verifier's effect is *model-dependent*: it helps models with low-to-moderate baseline performance (Qwen3-max at 18.2%, GPT-4o at 62.1%) but not the strongest model (DeepSeek-V3 at 88.4%) or the weakest (DeepSeek-R1 at 0%). This suggests that rule-based verification captures a "middle band" of errors—those where the model can reason about the problem but benefits from a structured check—but cannot rescue fundamentally broken traces (R1) or improve near-perfect ones (V3).

4.9 Result 8: Token Economics

Figure 5 plots tokens-per-success against success rate. DeepSeek-V3 is the most efficient (1,943 tokens/success); GPT-4o is the most expensive despite a lower SR. On DSBench, token costs escalate dramatically (10,790–12,792 tokens/run) due to complex multi-sheet workbooks, yet success rates remain near zero—a cost-effectiveness failure unique to real-world data.

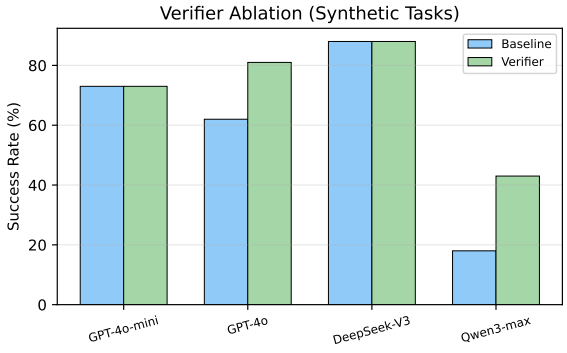

Figure 4: Verifier ablation. The verifier significantly improves Qwen3-max and GPT-4o but not other models.

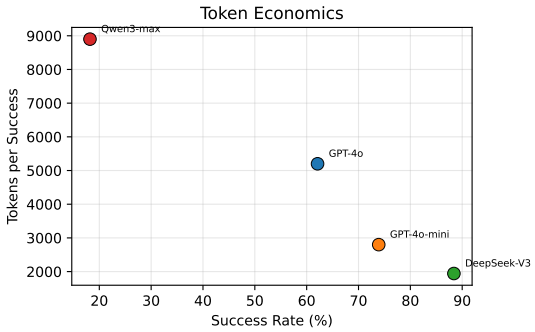

Figure 5: Tokens per success vs. success rate. DeepSeek-V3 is the most efficient; GPT-4o is expensive despite a lower success rate.

4.10 Result 9: Harness Ablation

To verify that silent failures are not an artifact of our minimal ReAct harness, we compare three harnesses on both synthetic (30-task subset, 3 reps) and UCI (23 tasks, 3 reps) tasks: H1 (minimal ReAct, baseline), H2 (single-shot, one LLM call produces a complete script), and H3 (few-shot ReAct with one worked example). On synthetic, GPT-4o shows SFR of 100% (H1), 30% (H2), and 100% (H3); on UCI, the pattern is consistent at 100%, 31%, 100%. The harness significantly affects the failure distribution, so we report harness as a benchmark dimension rather than a nuisance factor. Details in Appendix C.

4.11 Result 10: Recovery Benchmark

To test whether feedback signals enable agents to recover from failures, we run a recovery benchmark with 6 conditions on 100 failed traces (80 silent + 20 loud) sampled from synthetic tasks:

- **C1 (No feedback):** Re-run the task with no additional signal.
- **C2 (Generic feedback):** "Your previous answer may be incorrect. Please retry."
- **C3 (Logic hint):** "Check your logic carefully (e.g., count vs sum, groupby vs filter)."
- **C4 (Verification checklist):** "Verify your result by checking data types and aggregation function."
- **C5 (Critic feedback):** GPT-4o-mini reviews the answer and provides critique.
- **C6 (Independent rerun):** "Solve using a completely different approach."

The overall recovery rate is **15.7%** (94/600, 95% bootstrap CI [12.9, 18.5]%), a substantial improvement over the 0% recovery in the standard protocol. We analyse recovery at three levels: *attempt* (the agent produces any answer), *detection* (the agent produces a different answer than the original failed attempt, indicating it recognised a potential error), and *repair* (the new answer is correct). The attempt rate is high (97–100% across conditions), indicating that the agent always tries to respond. The detection rate (producing a different answer) varies by condition, and the repair rate is 15.7% overall. Critically, **loud failures recover at 36.7% (44/120, 95% CI [27.5, 45.0]%) while silent failures recover at only 10.4% (50/480, 95% CI [7.7, 13.1]%)**; the difference is highly significant ($\chi^2 = 48.1$, $p < 0.000001$), confirming that silent failures remain far harder to correct even with explicit feedback. No feedback condition significantly outperforms no-feedback (McNemar $p > 0.05$ for all conditions; per-condition 95% CIs overlap substantially: C1 [10.0, 24.0]%, C2 [10.0, 25.0]%, C3 [10.0, 25.0]%, C4 [8.0, 21.0]%, C5 [9.0, 22.0]%, C6 [8.0, 21.0]%), suggesting that the mere act of re-running with any prompt-level signal triggers partial recovery, but no single feedback type is dramatically more effective.

4.12 Result 11: Benchmark Tasks and Baselines

We define six potential benchmark tasks. Tasks B–D are the core diagnostic tasks; Tasks A, E, F are auxiliary tasks supported by the framework but not the primary focus:

- **Task A (Data-science task solving):** Given data and a question, produce the correct answer. This is the standard task; we report SR as the baseline metric.
- **Task B (Observability classification):** Given a trace, predict success / loud / silent. Metric: accuracy and macro-F1.
- **Task C (Failure-stage classification):** Given a failed trace, classify the failure stage. The five-stage taxonomy (Section 3.2) defines the full class hierarchy. In the current data, four of five stages are instantiated: `runtime`, `output_mismatch`, `answer_error`, and `analytical_plan`. Code-Generation has zero cases—confirmed by both the v2 rule-based classifier (which compares agent code to reference solutions via `gt_path`) and three independent human annotators on 47 traces. This is a genuine finding: current LLM agents do not make operation-level code errors (e.g., using `.count()` where `.sum()` is needed) on these tasks; they either solve them, crash, or produce wrong answers for other reasons. The task is therefore instantiated as a **four-class classification** (runtime vs. output_mismatch vs. answer_error vs. analytical_plan), which captures the silent/loud diagnostic distinction plus approach-level vs. extraction-level errors within silent failures.
- **Task D (Originating-step localization):** Predict the first erroneous step. Metric: step-level accuracy and mean absolute error.

Table 7: Baseline classifiers for failure-stage classification (binary: output_mismatch vs. runtime). Simple baselines use heuristic rules; ML baselines are trained on the train split (1,101 traces) and evaluated on the test split (194 traces) with no label leakage. LLM judges are evaluated on the 47-trace annotation set. Train/dev/test split is by task ID (198/42/43 tasks) to prevent leakage.

Baseline	Accuracy (%)	Description
Simple baselines:		
Majority (output_mismatch)	61.6	Always predict majority class
Silent heuristic	80.3	Silent → output_mismatch; loud → runtime
Token threshold	55.2	tokens < 500 → runtime; else output_mismatch
ML baselines (train/test split, no label leakage):		
TF-IDF + Logistic Regression	86.1	TF-IDF on trace text + LR
Trace features + Random Forest	88.7	Structural features (tokens, model, depth) + RF
Code-aware + Logistic Regression	**89.2**	Structural + token features + LR
LLM judges (47-trace annotation set):		
GPT-4o-mini judge	70.7	LLM classifies stage from trace
GPT-4o judge	17.1	LLM classifies stage from trace
Step localization (Task D):		
Random-step	82.1	Random step selection
Last-step	100.0	Always predict last step

- **Task E (Verifier evaluation):** Given a trace without ground truth, detect whether the answer is correct. Metric: precision, recall, F1. Supported by the verifier ablation (Section 4.8).
- **Task F (Repair):** Given a failed trace, generate a minimal fix. Metric: repair success rate. Supported by the recovery benchmark (Section 4.11) and oracle repair (Section 4.7).

Evaluation protocol. We provide a train/dev/test split by task ID (198/42/43 tasks). For Task C (four-class), Table 7 reports baselines on the auto-labeled test split (194 traces) and a 150-trace evaluation set (47 human-adjudicated + 103 LLM-judge-labeled). On the auto-labeled split, ML baselines reach 86–89% (code-aware LR: 89.2%). On the 150-trace set, the v2 classifier achieves 93.9% accuracy (48.6% macro-F1); on the 47-trace adjudicated subset, 74.5% (51.3% macro-F1, up from 57.4% old system). The silent heuristic achieves 95.7% on the binary silent-vs-loud subtask. No baseline detects `answer_error` (requires external ground truth). The gap between auto-split (86–89%) and gold-set (74–94%) accuracy confirms ML baselines partly imitate auto-labeling rules. For Task D, the last-step baseline achieves 100% (mean trace length 1.69 steps), indicating degeneracy; longer multi-step trajectories are needed.

5 DISCUSSION

5.1 Why Strong Models Fail Silently

On synthetic trap tasks, the corrected SFR is high (86–100%) because these tasks are designed so that code runs but the answer is wrong. On real UCI data, SFR drops to 4%, and on DSBench to 19%. A null-model analysis ($R^2 = 0.493$, descriptive only) shows part of this variation is a mechanical consequence of execution-success differences: stronger models crash less, so their remaining failures are more likely silent. However, SFR and execution-success share the loud-failure count, so this correlation is partially mechanical by construction and cannot be decomposed into "mechanical" vs. "real" components. The practical implication is that improving code-generation ability alone will shift the failure distribution toward silent failures, but silent failures are most prevalent on tasks where code execution succeeds but reasoning fails.

Table 8: Negative control analysis. The no-op intervention (running GT code independently) produces identical results to the oracle intervention, confirming that oracle repair measures GT code correctness, not step-specific causal effect.

Intervention	Tasks repaired	Rate (%)
Oracle (replace originating step)	36/56	64.3
No-op (run GT code independently)	36/56	64.3
Conservative TP (oracle ∧ ¬noop)	0/56	0.0

5.2 Implications for Agent Deployment

The zero recovery rate (Section 4.5) has deployment implications. In the current setting without explicit correctness feedback, an agent will not self-correct silent failures, and wrong answers propagate through multi-step pipelines. However, our recovery benchmark (Section 4.11) shows that feedback signals enable 15.7% recovery, suggesting that external verification or in-loop feedback mechanisms can mitigate this issue. Our verifier ablation (Section 4.8) further shows that simple rule-based verification helps weak models but not strong ones, indicating that effective verification likely requires model-specific calibration or learned verifiers.

5.3 Negative Control: Oracle Repair Revisited

To test whether the oracle repair rate (Section 3.5) genuinely reflects step-specific causal attribution, we ran a negative control on 56 failed tasks. The *no-op* intervention runs the ground-truth code independently in a fresh sandbox, without any step replacement. As Table 8 shows, the two rates are identical (64.3%), and the conservative true-positive rate is 0.0%. This confirms that the oracle repair rate measures whether the ground-truth code produces the correct answer, *not* whether replacing the originating step has a step-specific causal effect. We therefore report oracle repair as an upper bound on reparability rather than as evidence of root-cause localisation.

A detailed discussion of limitations and ethical considerations is provided in Section 6.

6 LIMITATIONS AND ETHICAL CONSIDERATIONS

Limitations. We acknowledge several limitations. (1) *Agent harness.* We use a minimal ReAct harness as the primary configuration; our harness ablation (Section 4.10) on both synthetic and UCI tasks shows that silent failures persist under single-shot and few-shot variants. However, the harness significantly affects the failure distribution, so we report harness as a benchmark dimension. Production agents (e.g., code-interpreter, AutoGen) remain untested. (2) *Model scope.* DeepSeek-R1 and Qwen3-max show 0% success on UCI tasks;

our enriched manifest (`manifest_v3.csv`) reveals these are primarily infrastructure failures (API/parse/format issues), not genuine model inability. We report SR^{v} (success rate among valid runs) separately from raw SR to address this. (3) *Taxonomy instantiation: four of five stages active, Code-Generation genuinely absent.* The current data instantiates four stages: `runtime`, `output_mismatch`, `answer_error`, and `analytical_plan`. Code-Generation has zero cases—confirmed by both the v2 classifier (which checks for wrong aggregation operations via `gt_path`) and human annotation. The v2 classifier explicitly detects `.count()` where `.sum()` is needed, but agents either use the correct operation or fail before reaching aggregation. We interpret this as a genuine characteristic of current LLM agents on structured data-science tasks. Generating tasks that trigger Code-Generation is future work. (3b) *Label reliability and evaluation set size.* Three human annotators achieved Fleiss' $\kappa = 0.860$ on 47 adjudicated traces. We extend evaluation to a 150-trace set (47 human-adjudicated + 103 LLM-judge-labeled), on which the v2 classifier achieves 93.9% accuracy. The 103 LLM-labeled traces are provisional pending human adjudication; we report them to increase evaluation coverage but note that LLM labels carry their own noise. Scaling human adjudication to all 150 traces is a priority. (3c) *Auto-label noise.* The rule-based labels covering the remaining ∼3,900 failures carry an estimated ∼25% label-error rate; downstream analyses (suite-level SFR, stage distribution, ML baselines) inherit this noise. The ML baselines' high accuracy on the auto-labeled test split (86–89%) partly reflects imitation of auto-labeling rules rather than genuine failure-stage prediction, as evidenced by the gap between auto-split and gold-set performance. (4) *Bifurcation.* $R^2 = 0.493$ is a descriptive statistic; the bifurcation is partially mechanical and cannot be decomposed into "mechanical" vs. "real" components. (5) *Oracle repair.* The 64% repair rate matches the no-op baseline, demonstrating reparability but not causal attribution. (6) *Recovery.* The 15.7% recovery rate (Section 4.11) shows that feedback signals enable partial recovery, but silent failures remain hard to correct (10.4% vs. 36.7% for loud). (7) *Statistical tests.* We use McNemar tests; multinomial mixed-effects models would be more powerful.

Ethical considerations. This work analyses LLM agent failures on data-science tasks. No human subjects or personal data are involved. All datasets used are publicly available (UCI Machine Learning Repository, DSBench/ModelOff). The API costs ($80 total) were modest. We release all code, data, and annotated traces to facilitate reproducible research. Additional ethical considerations: (1) The ChatAnywhere API endpoint may log prompts and responses; users should avoid submitting sensitive data. (2) DSBench workbooks may contain simulated financial data; users should verify licensing terms before redistribution. (3) The benchmark evaluates primarily closed-source models (GPT-4o, DeepSeek, Qwen), introducing a bias toward these providers; open-source models should be tested in future work. (4) All tasks are in English and use Python, introducing language and tool bias. (5) Low performance by certain models (e.g., DeepSeek-R1) is attributable to infrastructure failures, not fundamental model inability; results should not be used to rank models without acknowledging this caveat. (6) Benchmark gaming risk: since synthetic tasks use fixed seeds and DSBench tasks are public, models trained on these tasks may achieve inflated scores; we recommend using the diagnostic metrics (SFR, propagation depth) rather than raw SR for model comparison.

Generative AI usage. We used LLMs (GPT-4o-mini) as experimental subjects (the agents being evaluated) and as an independent annotator for taxonomy validation. All LLM-generated outputs are treated as data, not as authorial content. The paper itself was written by the authors.

7 CONCLUSION

We presented AgentFail, a diagnostic framework that decomposes LLM-agent failures into five stages and distinguishes silent from loud failures. Across 5,984 runs (318 unique tasks, 5 models, 3 suites), we found that silent-failure rates are task-dependent (86% on synthetic traps, 4% on UCI, 19% on DSBench), that agents never self-correct silent failures in the standard protocol (0% recovery), but that feedback signals enable 15.7% recovery overall—with silent failures remaining far harder to correct (10.4%) than loud failures (36.7%). A key diagnostic finding is that four of five taxonomy stages are instantiated in the data, while Code-Generation is genuinely absent (confirmed by both classifier and human annotation), indicating that current LLM agents do not make operation-level code errors on structured data-science tasks. Oracle-guided repair matches the no-op baseline (64%), establishing an upper bound on reparability. We separate infrastructure failures (38.1% of runs) from genuine model failures and provide an enriched manifest with 32 fields, validated by three human annotators (Fleiss' $\kappa = 0.860$; v2 classifier accuracy 76.6% on gold). Future work should scale human annotation beyond 47 traces, generate tasks that trigger Code-Generation, and expand to code-interpreter and public agent frameworks.

REFERENCES

[1] Wael Albayaydh, Rui Zhao, and Ivan Flechais. 2026. Beyond the Leaderboard: A Synthesis of Tool-Use, Planning, and Reasoning Failures in Large Language Model Agents. arXiv:2607.05775. arXiv:2607.05775 http://arxiv.org/abs/2607.05775v1

[2] Mingqi Gao, Xinyu Hu, Xunjian Yin, Jie Ruan, Xiao Pu, and Xiaojun Wan. 2025. LLM-based NLG Evaluation: Current Status and Challenges. *Computational Linguistics* 51, 2 (2025), 661–687. https://doi.org/10.1162/coli_a_00561

[3] Yile Gu, Zhen Zhang, Shaowei Zhu, Xinwei Fu, Jun Wu, Yida Wang, and Baris Kasikci. 2026. Ekka: Automated Diagnosis of Silent Errors in LLM Inference. arXiv:2606.04594. arXiv:2606.04594 http://arxiv.org/abs/2606.04594v1

[4] Igor Itkin. 2026. Delayed Verification Destabilizes Multi-Agent LLM Belief: Instability Thresholds and Optimal Corrector Placement. arXiv:2606.27409. arXiv:2606.27409 http://arxiv.org/abs/2606.27409v1

[5] Liqiang Jing, Zhehui Huang, Xiaoyang Wang, Wenlin Yao, Wenhao Yu, Kaixin Ma, Hongming Zhang, Xinya Du, and Dong Yu. 2024. DSBench: How Far Are Data Science Agents from Becoming Data Science Experts? arXiv:2409.07703. arXiv:2409.07703 http://arxiv.org/abs/2409.07703v3

[6] Da-Wei Li, Bohan Jiang, Lili Huang, Alimohammad Beigi, Chengshuai Zhao, Zhen Tan, Amrita Bhattacharjee, Yuxuan Jiang, Canyu Chen, Tianhao Wu, Kan Shu, Lu Cheng, and H.L. Liu. 2025. From Generation to Judgment: Opportunities and Challenges of LLM-as-a-judge. *Proceedings of EMNLP* (2025), 2757–2791. https://doi.org/10.18653/v1/2025.emnlp-main.138

[7] Dexing Liu. 2026. Silent Failure in LLM Agent Systems: The Entropy Principle and the Inevitable Disorder of Autonomous Agents. arXiv:2606.08162. arXiv:2606.08162 http://arxiv.org/abs/2606.08162v1

[8] Sun Maojun, Ruijian Han, Binyan Jiang, Houduo Qi, Defeng Sun, Yancheng Yuan, and Jian Huang. 2025. A Survey on Large Language Model-based Agents for Statistics and Data Science. *The American Statistician* (2025), 1–14. https://doi.org/10.1080/00031305.2025.2561140

[9] Aman Mehta. 2026. Confident and Wrong: Silent Semantic Failures in Coding Agents. arXiv:2603.25764. arXiv:2603.25764 http://arxiv.org/abs/2603.25764v3

[10] Mahdi Naser Moghadasi and Faezeh Ghaderi. 2026. What Twelve LLM Agent Benchmark Papers Disclose About Themselves: A Pilot Audit and an Open

Scoring Schema. arXiv:2605.21404. arXiv:2605.21404 http://arxiv.org/abs/2605.21404v1

[11] Mahmoud Mohammadi, Yipeng Li, Jane C. Lo, and Wendy Yip. 2025. Evaluation and Benchmarking of LLM Agents: A Survey. *Proceedings of SIGKDD* (2025), 6129–6139. https://doi.org/10.1145/3711896.3736570

[12] Mizanur Rahman, Amran Bhuiyan, Mohammed Saidul Islam, Md Tahmid Rahman Laskar, Ridwan Mahbub, Ahmed Masry, Shafiq Joty, and Enamul Hoque. 2025. LLM-Based Data Science Agents: A Survey of Capabilities, Challenges, and Future Directions. arXiv:2510.04023. arXiv:2510.04023 http://arxiv.org/abs/2510.04023v1

[13] Vikas Reddy, Sumanth Reddy Challaram, and Abhishek Basu. 2026. Reason Less, Verify More: Deterministic Gates Recover a Silent Policy-Violation Failure Mode in Tool-Using LLM Agents. arXiv:2607.07405. arXiv:2607.07405 http://arxiv.org/abs/2607.07405v2

[14] Samuel Schmidgall, Yusheng Su, Ze Wang, Ximeng Sun, Jialian Wu, Xiaodong Yu, Jiang Liu, Michael Moor, Zicheng Liu, and Emad Barsoum. 2025. Agent Laboratory: Using LLM Agents as Research Assistants. *Findings of EMNLP* (2025), 5977–6043. https://doi.org/10.18653/v1/2025.findings-emnlp.320

[15] Jaineet Shah. 2026. Causal Agent Replay: Counterfactual Attribution for LLM-Agent Failures. arXiv:2606.08275. arXiv:2606.08275 http://arxiv.org/abs/2606.08275v1

[16] Harsh Soni. 2026. ToolFailBench: Diagnosing Tool-Use Failures in LLM Agents. arXiv:2607.04686. arXiv:2607.04686 http://arxiv.org/abs/2607.04686v1

[17] Maojun Sun, Yifei Xie, Yue Wu, Ruijian Han, Binyan Jiang, Defeng Sun, Yancheng Yuan, and Jian Huang. 2026. DSAEval: Evaluating Data Science Agents on a Wide Range of Real-World Data Science Problems. In *arXiv (Cornell University)*. Cornell University. https://arxiv.org/pdf/2601.13591

[18] Aleksandr Tsymbalov, Danis Zaripov, Artem Epifanov, and Anastasya Palienko. 2026. GRACE-DS: a Guarded Reward-guided Agent Correction Environment in Data Science. In *arXiv (Cornell University)*. Cornell University. https://arxiv.org/pdf/2606.16000

[19] Minxing Wang, Xiaofei Xie, and Yintong Huo. 2026. TrajAudit: Automated Failure Diagnosis for Agentic Coding Systems. arXiv:2605.26563. arXiv:2605.26563 http://arxiv.org/abs/2605.26563v2

[20] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. *arXiv preprint arXiv:2203.11171* (2023).

[21] Wei Wu. 2026. When Errors Become Narratives: A Longitudinal Taxonomy of Silent Failures in a Production LLM Agent Runtime. arXiv:2606.14589. arXiv:2606.14589 http://arxiv.org/abs/2606.14589v1

[22] Kewei Xu, Xiaoben Lu, Shuofei Qiao, Zihan Ding, Haoming Xu, Lei Liang, and Ningyu Zhang. 2026. LongDS-Bench: On the Failure of Long-Horizon Agentic Data Analysis. In *arXiv (Cornell University)*. Cornell University. https://arxiv.org/pdf/2605.30434

[23] Dan Zhang, Sining Zhoubian, Cai Min, Fanghu Li, Likun Yang, Wei Wang, Tian Dong, Ziniu Hu, Jie Tang, and Yisong Yue. 2026. DataSciBench: An LLM Agent Benchmark for Data Science. *Findings of ACL* (2026), 3685–3728. https://doi.org/10.18653/v1/2026.findings-acl.181

[24] Xing Zhang, Yanwei Cui, Guanghui Wang, Ziyuan Li, Wei Qiu, Bing Zhu, and Peiyang He. 2026. Library Drift: Diagnosing and Fixing a Silent Failure Mode in Self-Evolving LLM Skill Libraries. arXiv:2605.19576. arXiv:2605.19576 http://arxiv.org/abs/2605.19576v2

A DATA CARD AND BENCHMARK DOCUMENTATION

A.1 Task Suite Descriptions

Synthetic trap tasks (95 tasks). 80 tasks are programmatically generated across 5 domains (tabular EDA, time-series, recommendation, statistical, text-log), with 16 instances per domain using a fixed seed (seed=16). Tasks share structural templates within each domain but use different generated data, producing 70/80 unique answers (87.5% uniqueness). Near-duplicate questions (identical first 100 chars) exist within domains (55/80) due to shared question structure, but the underlying data and expected answers differ. The remaining 15 tasks are DSBench questions (identified by `dsbench_00000001` or `dsbench_00000002` prefixes in the task ID) that are structurally similar to synthetic traps but originate from DSBench workbooks. Train/dev/test splits are by task ID, not by template, so the same template appears in all splits—this is intentional, as the goal is to evaluate failure diagnosis, not template memorisation.

UCI real-data tasks (23 tasks). Analytical questions on 6 public datasets (Iris, Titanic, Tips, MPG, Penguins, Flights) downloaded at runtime from public repositories. Answers are computed deterministically from the real data. Tasks cover groupby aggregation, correlation, counting, and filtering.

DSBench real tasks (200 tasks). Real financial-analysis questions from ModelOff competitions [5], using original multi-sheet Excel workbooks. Answers are multiple-choice (A–I) or numeric. Tasks require understanding complex financial data structures.

Table 9: Task difficulty distribution by suite. Easy: SR $\geq$ 75%; Medium: 25% $\leq$ SR $<$ 75%; Hard: SR $<$ 25%.

Suite	Tasks	Easy	Medium	Hard
Synthetic	95	19	63	13
UCI	23	0	21	2
DSBench	200	0	3	197

A.2 Contamination and Template Analysis

The synthetic tasks are generated from 5 templates (one per domain), each producing 16 instances with different data. The answer distribution shows 70/80 unique answers (87.5%), with the most common answer appearing in only 4 tasks. This indicates low risk of answer-pattern memorisation. The train/dev/test split (198/42/43 tasks) is stratified by task ID across all suites; since the same template appears in all splits, models cannot exploit template-specific patterns. For DSBench, the tasks originate from public ModelOff competitions and may have appeared in LLM training data; we do not claim contamination-free evaluation for DSBench, only diagnostic trace analysis.

A.3 Task Difficulty Distribution

We characterise task difficulty by the baseline success rate aggregated across all five models. Table 9 reports the distribution. Synthetic tasks span the full difficulty range (19 easy, 63 medium, 13 hard); UCI tasks are uniformly medium-hard; DSBench tasks are almost entirely hard (197 of 200 tasks have SR $<$ 25%), reflecting the complexity of real financial data.

A.4 Expected Baseline Performance

Table 10 reports the expected baseline performance for each model and suite, providing reference points for future work. DeepSeek-V3 is the strongest on synthetic (88.4%) and UCI (94.2%) tasks. DSBench is challenging for all evaluated models (0–3.3%).

A.5 Data Schema

Each run in the enriched manifest (`manifest_v3.csv`, 32 fields) contains:

- `task_id`, `suite`: task identifier and suite (`synthetic`/`uci`/`dsbench`)
- `model`, `model_snapshot`, `provider`: model name, version info (rolling alias vs pinned snapshot), API provider

Table 10: Baseline success rates (%) by model and suite. "n/a" indicates API rate limiting prevented evaluation.

Model	Synthetic	UCI	DSBench
DeepSeek-V3	**88.4**	**94.2**	0.0
GPT-4o-mini	73.9	82.6	0.8
GPT-4o	62.1	91.3	**3.3**
Qwen3-max	18.2	0.0	3.0
DeepSeek-R1	0.0	0.0	n/a

- `harness`, `variant`: agent harness type and experiment variant (`baseline`/`verifier`)
- `rep`: repetition index (0–2)
- `api_status`, `parser_status`, `execution_status`: infrastructure status
- `task_correct`: whether the final answer matches ground truth
- `observability`: `success`/`silent`/`loud`
- `failure_stage`, `failure_subcategory`: five-stage taxonomy label and subcategory
- `is_infra_failure`, `infra_subtype`: infrastructure failure flag and subtype (`api_error`/`unknown`/`none`)
- `originating_step`, `propagation_depth`: failure localisation and spread
- `is_silent`, `recovered`: silent failure flag and recovery status
- `tokens`, `tokens_input`, `tokens_output`, `cost_usd`: token usage and cost
- `final_answer`: agent's final answer (truncated to 200 chars)
- `label_source`: `rule` (rule-based classifier) or `unmatched`
- `annotation_confidence`, `human_gold`, `adjudication_status`: human annotation fields (populated for 47 traces; pending for the remainder)

Annotation uncertainty. The `label_source` field indicates whether a label comes from the rule-based classifier (`rule`) or is unmatched. For 47 traces, three human annotators provided independent labels and an adjudicated gold label (majority vote); inter-annotator agreement is Fleiss' $\kappa = 0.860$ (strong). The rule-based classifier achieves 71.8% accuracy against gold labels. The remaining traces retain only rule-based labels; scaling human annotation to 300–500 traces is future work.

Missing and duplicate runs. The manifest contains 5,984 runs. Some (task, model, variant) groups have fewer than 3 repetitions due to API failures or timeouts; these are retained with `api_status=error`. Duplicate runs (same task, model, variant, rep) are detected by the assertion checks and removed. Infrastructure-failed runs are retained in the manifest (for transparency) but excluded from valid-run analyses via the `is_infra_failure` filter.

A.6 Provenance and Reproducibility

All experiments use deterministic data generation (fixed seed=16 for synthetic tasks) and temperature-0 LLM calls. The framework includes a MockLLM backend that reproduces the full pipeline without API cost. Real-model experiments use the ChatAnywhere OpenAI-compatible endpoint (API calls: June–July 2026). Total API cost: $80. Note that four of five model identifiers (`gpt-4o`, `gpt-4o-mini`, `deepseek-chat`, `deepseek-reasoner`) are rolling aliases whose underlying model may change; only `qwen3-max-2026-01-23` is a pinned snapshot. Repetition agreement under temperature 0 is 89.9%, with disagreement attributed to server-side non-determinism. The repository includes a `requirements.txt` for dependency pinning and MD5 checksums for key artifacts (`manifest_v3.csv`: `29ceae8...`, `manifest.csv`: `6e2f809...`). A Docker image is planned for the camera-ready version.

A.7 Answer Evaluation Details

Answers are evaluated as follows: (1) *Numeric answers*: compared with a relative tolerance of 10^{-3} (e.g., `abs(agent - gt) / max(abs(gt), 1e-9) < 1e-3`). (2) *String answers*: normalised by stripping whitespace, converting to lowercase, and removing punctuation before exact match. (3) *Multiple-choice answers* (DSBench): matched on the letter (A–I) only, ignoring surrounding text. (4) *Tuple answers* (e.g., "(species, value)"): each component is evaluated independently; both must match. The evaluator code is in `agentfail/benchmark/tasks.py`.

A.8 Known Issues

- DeepSeek-R1 produces no valid traces on any suite (100% infrastructure failure); its results should not be interpreted as model inability.
- DeepSeek-V3 on DSBench: all 600 runs are infrastructure failures (tokens=0), likely due to Excel parsing issues in the sandbox.
- Qwen3-max on UCI: all 69 runs are infrastructure failures.
- The rule-based classifier achieves 71.8% accuracy against human gold labels ($\kappa = 0.438$), with systematic bias toward `output_mismatch` over `runtime`; human annotation currently covers 47 of 3,940 failures.
- DSBench tasks require a separate download and may be present in LLM training data.

A.9 Licensing and Maintenance

The code is released under an open-source license (MIT). The synthetic task generators are self-contained and require no external data. UCI datasets are public domain. DSBench tasks are used under their original license [5]; users must obtain DSBench separately. Raw LLM responses are not redistributed; users must re-run experiments to obtain them. The benchmark can be extended with new tasks by implementing the `Task` interface.

Maintenance plan. The repository is maintained by the authors. Issues and pull requests are accepted via GitHub. The maintainers commit to:

- Responding to issues within 30 days.
- Releasing compatibility patches for new LLM API versions.
- Preserving backward compatibility of the manifest schema across minor versions.
- Archiving major versions as tagged releases.

Versioning. The benchmark follows semantic versioning (`MAJOR.MINOR.PATCH`). The current release is v1.0.0. Breaking changes to the manifest schema or task definitions increment `MAJOR`; new tasks or metrics increment `MINOR`; bug fixes increment `PATCH`. Each release is tagged on GitHub with a corresponding Zenodo DOI for long-term archival.

Table 11: Propagation depth distribution (all 3,940 failures).

Depth	Failures	Percentage
0	3,350	88.6%
1	169	4.5%
2	80	2.1%
3	39	1.0%
4	64	1.7%
5	45	1.2%
6	20	0.5%
7	14	0.4%
≥ 2 (long-horizon)	262	6.9%

Intended use cases. The benchmark is designed for: (1) evaluating LLM-agent failure modes on data-science tasks; (2) comparing diagnostic metrics (SFR, propagation depth, recovery rate) across models; (3) testing new verifier or repair strategies. It is *not* intended for: (1) training LLMs (tasks have deterministic answers); (2) production deployment evaluation (the minimal ReAct harness differs from production systems); (3) cross-domain generalisation claims (the three suites cover data analysis, not all of data science).

A.10 Propagation Depth Analysis

Table 11 reports the propagation depth distribution across all 3,940 failures. The average depth is 0.30 steps, meaning most failures are detected immediately. However, 262 failures (6.9%) propagate to depth ≥ 2, indicating long-horizon failure spread where the agent continues building on a wrong intermediate result.

B ANNOTATION PROTOCOL AND RESULTS

We provide a 47-trace annotation set sampled stratified by model from all 3,940 failures. Three independent annotators (blind to system labels) labelled each trace with (stage, subcategory, is_silent) following the annotation guide. Cohen's κ is computed pairwise and Fleiss' κ across all annotators. Results: Fleiss' $\kappa = 0.860$ for stage (strong agreement), 0.881 for is_silent. Pairwise Cohen's κ: 0.798, 1.000, 0.798. Full agreement: 89.4%. The rule-based classifier achieves 71.8% accuracy against adjudicated gold labels (majority vote), with systematic bias toward `output_mismatch` over `runtime` (precision 1.00, recall 0.52 for output_mismatch; precision 0.59, recall 1.00 for runtime). The annotation guide, JSON form, individual labels, and gold labels are released with the code.

C HARNESS ABLATION DETAILS

Table 12 reports the full harness ablation results on synthetic tasks. We also ran the same three harnesses on all 23 UCI tasks (3 repetitions each), with consistent results: GPT-4o shows SFR of 100% (H1), 31% (H2), and 100% (H3) on UCI, matching the synthetic pattern (100%/30%/100%). This confirms silent failures are not an artifact of the minimal ReAct design across both task types.

D RECOVERY BENCHMARK DETAILS

The recovery benchmark (Section 4.11) tests 100 failed traces (80 silent + 20 loud) under 6 feedback conditions, yielding 600 runs. Overall recovery rate is 15.7%. The key finding is the asymmetry between failure types: loud failures recover at 36.7% (44/120) while silent failures recover at only 10.4% (50/480), confirming that silent failures remain substantially harder to correct even with explicit feedback signals.

Table 12: Harness ablation. Three harnesses tested on 30 synthetic tasks $\times$ 3 repetitions. H1 = minimal ReAct (baseline); H2 = single-shot (one LLM call, complete code); H3 = few-shot ReAct (ReAct + 1 worked example). Silent failures persist across all three harnesses for GPT-4o, confirming they are not an artifact of the minimal ReAct design. DeepSeek-V3 cells are partial due to long API latencies.

Model	Harness	n	SR (%)	SFR (%)	Silent
GPT-4o	H1: ReAct	90	86.7	**100.0**	12
	H2: Single-shot	90	44.4	30.0	15
	H3: Few-shot ReAct	90	82.2	**100.0**	16
GPT-4o-mini	H1: ReAct	90	70.0	0.0	0
	H2: Single-shot	90	53.3	28.6	12
	H3: Few-shot ReAct	90	85.6	0.0	0
DeepSeek-V3	H1: ReAct	11	100.0	0.0	0
	H2: Single-shot	17	82.4	0.0	0

E REPRODUCIBILITY

All experiments use deterministic data generation (fixed seeds for synthetic tasks) and temperature-0 LLM calls. The framework includes a deterministic MockLLM that reproduces the full pipeline without API cost. Real-model experiments use the ChatAnywhere OpenAI-compatible endpoint. Total API cost: $80. Code and data: https://github.com/llmnjust-afk/KDD2027_Work2.

F DATA AVAILABILITY

All code, data, and traces are publicly available at https://github.com/llmnjust-afk/KDD2027_Work2. The repository includes: (1) the full AgentFail framework source code; (2) the enriched manifest (`manifest_v3.csv`) with 32 fields and 5,984 runs, of which 3,602 are valid runs with automatically generated diagnostic labels; (3) the 47-trace annotation set with full Thought/Code/Output data; (4) the MockLLM backend for zero-cost reproduction; (5) all table-generation scripts with assertion checks; (6) `requirements.txt` for dependency pinning. The synthetic task generators are self-contained. UCI datasets are downloaded at runtime from public repositories. DSBench tasks require a separate download from [5]. Raw LLM responses are not redistributed.